On All-Substrings Alignment Problems

Wei Fu[1], Wing-Kai Hon[2], and Wing-Kin Sung[1]*

[1] School of Computing, National University of Singapore, Singapore,
{fuwei,ksung}@comp.nus.edu.sg,
[2] Department of Computer Science and Information Systems, The University of Hong Kong, Hong Kong,
wkhon@csis.hku.hk

Abstract. Consider two strings A and B of lengths n and m respectively, with $n \ll m$. The problem of computing global and local alignments between A and all m^2 substrings of B can be solved by the classical Needleman-Wunsch and Smith-Waterman algorithms, respectively, which takes $O(m^2n)$ time and $O(m^2)$ space. This paper proposes faster algorithms that take $O(mn^2)$ time and $O(mn)$ space. The improvement stems from a compact way to represent all the alignment scores.

1 Introduction

Sequence comparisons have been studied extensively for the last few decades [4, 5]. Many useful metrics are defined, which find applications in various areas. *Alignment score* is perhaps the most popular metric in the literature. Examples of its usages can be found in text processing, such as syntax error correction [1] and spelling correction [3], or in comparing graphs [6].

Due to the breakthrough in bio-technology in recent decades, biological sequences such as DNA, RNA or protein, are being rapidly produced in the laboratories everyday. Sequence comparison now plays an important role in extracting useful information from these raw biological data. Scientists are often interested in comparing a short sequence (usually, a known gene or a motif) with many substrings in a long sequence (usually, a genome). In such scenario, it is good for us to perform a preprocessing so that the above kind of comparison between the two strings can be done efficiently afterwards. This motivates the All-Substrings Alignment Problem: Given two strings A and B of lengths n and m respectively, with $n \ll m$, the problem is to find the alignment scores between A and all substrings $B[i..j]$. The alignment can be global alignment, suffix alignment, local alignment, or semi-global alignment.

A naive solution to this problem is to compute the alignment scores between A and all substrings of B explicitly using the classical Needleman-Wunsch and Smith-Waterman algorithms [7,9]. Such approach requires $O(m^2n)$ time and $O(m^2)$ space. This paper solves these problems by devising an $O(mn^2)$-time algorithm. This is significant since n is much smaller than m.

* Research supported in part by NUS Academic Research Grant R-252-000-119-112.

T. Warnow and B. Zhu (Eds.): COCOON 2003, LNCS 2697, pp. 80–89, 2003.

For global alignment, our improvement is from the observation that, although the alignments between A and different substrings of B vary, their scores can actually be 'compressed' under suitable transformation. Precisely, for any fixed prefix $A[1..k]$, the global alignment scores between $A[1..k]$ and all substrings of B can be stored in $O(mn)$ space instead of the trivial $O(m^2)$ space. We also show that this compressed structure for $A[1..k]$ can be computed directly from that for $A[1..k-1]$. These ideas lead to our algorithm, which is incremental in nature. Moreover, the compressed structures themselves can serve as a compact index for storing all the global alignment scores between A and any substring of B, so that retrieving a particular value takes $O(\log n)$ time.

For local, semi-global, and suffix alignments, we observe that among the m^2n alignments between $A[1..k]$ and $B[i..j]$ for all $1 \leq k \leq n$ and $1 \leq i < j \leq m$, many of them are the same. More precisely, there are at most mn^2 useful values, which can be found in $O(mn^2)$ time. To retrieve a particular alignment score between $A[1..k]$ and $B[i..j]$, we only need to do a range maximum query over these $O(mn^2)$ value.

The organization of the paper is as follows. Section 2 gives basic definitions of the problems. Section 3 describes the $O(mn^2)$-time algorithm for solving the All-Substrings Global Alignment Problem, while the other alignment problems are discussed in Section 4. Finally, Section 5 concludes with some open problems.

2 Preliminaries

We formally define some notations which are used throughout the paper.

Definition 1. [8] *An* alignment *between two sequences is formed by the insertion of spaces in arbitrary locations along the sequences so that they end up with the same length.*

Having the same length, the augmented sequences of an alignment can be placed one over the other, creating a mapping between characters or spaces in the first sequence and characters or space in the second sequence. In general, we assume that no space in one sequence is aligned with a space in another sequence.

To measure the similarity of an alignment, we need the concept of *score scheme*.

Definition 2. *Let Σ denote a set of characters and let Σ' denote $\Sigma \cup \{\sqcup\}$, where the symbol $\sqcup$ represents a space. A* score scheme *over Σ is a function $\delta : \Sigma' \times \Sigma' \to \mathbb{Q}$, where $\mathbb{Q}$ denotes the set of rational numbers.*

Given a score scheme, the *score* of an alignment is defined as follows.

Definition 3. *The* score *of an alignment is the summation of the δ function over every pair of the corresponding characters in the given alignment.*

We now define the problems that we are interested in this paper. Let $A[1..n]$ and $B[1..m]$ be two sequences of characters over an alphabet Σ.

Definition 4. *The* global alignment score *between two sequences is defined as the score of the highest scoring alignment between the two sequences. The* All-Substrings Global Alignment Problem on (A,B) *is to find the global alignment scores between A and any substring of B.*

Definition 5. *The* suffix alignment score *between two sequences is defined as the score of the highest scoring alignment between a suffix of one sequence and a suffix of the other one. The* All-Substrings Suffix Alignment Problem on (A,B) *is to find the suffix alignment scores between A and any substring of B.*

Definition 6. *The* semi-global alignment score *between two sequences S and T is defined as the score of the highest scoring alignment between S and a substring of T. The* All-Substrings Semi-global Alignment Problem on (A,B) *is to find the semi-global alignment scores between A and any substring of B.*

Definition 7. *The* local alignment score *between two sequences is defined as the score of the highest scoring alignment between a substring of one sequence and a substring of the other one. The* All-Substrings Local Alignment Problem on (A,B) *is to find the local alignment scores between A and any substring of B.*

3 All-Substrings Global Alignment

In this section, we consider the All-Substrings Global Alignment Problem on (A, B) between two sequences $A[1..n]$ and $B[1..m]$. We first assume the followng score scheme.

1. $\delta(x, \sqcup) = \delta(\sqcup, x) = -1$.
2. $\delta(x, y) = \begin{cases} 1, & \text{if } x = y \text{ and } x, y \neq \sqcup \\ -1, & \text{if } x \neq y \text{ and } x, y \neq \sqcup \end{cases}$

Let $H_k[i, j]$ denote the global alignment score between a prefix $A[1..k]$ of A and a substring $B[i..j]$ of B. Our problem is thus equivalent to finding all the values $H_n[i, j]$ for every $1 \leq i \leq j \leq m$. This can be solved based on the following lemma.

Lemma 1.

$$H_k[i, j] = \max \begin{cases} H_k[i, j-1] & + \delta(\sqcup, B[j]) \\ H_{k-1}[i, j] & + \delta(A[k], \sqcup) \\ H_{k-1}[i, j-1] & + \delta(A[k], B[j]) \end{cases}$$

The framework of computing H_n is simple: First compute H_0, then complete the matrices H_k for $k = 1, 2, \ldots, n$. Note that each H_k has $O(m^2)$ entries, and each entry can be filled in $O(1)$ time by Lemma 1. Thus, we get H_n in $O(m^2 n)$ time. For the space, it takes $O(m^2)$, since to compute any matrix H_k, we only need to keep H_{k-1}.

When m is very large (which is common in the problem we are modeling), the above time and space complexities are not practical. We propose an alternative way to solve this problem in $O(mn^2)$ time and $O(mn)$ space, making use of a simple modification on the matrices. The next section shows such modification, while the details of the new algorithm is discussed afterwards.

3.1 Transformation of H_k's

Instead of computing H_k's, we compute another set of matrices F_k's as follows.

Definition 8. $F_k[i,j] = H_k[i,j] + j - i + 1 + k$.

Similar to Lemma 1, we have a corresponding formula that relates the adjacent matrices F_{k-1} and F_k.

Fact 1.

$$F_k[i,j] = \max \begin{cases} F_k[i,j-1] \\ F_{k-1}[i,j] \\ F_{k-1}[i,j-1] \ + \ 2 \ + \ \delta(A[k],B[j]) \end{cases}$$

Note that $H_k[i,j]$ can be retrieved in constant time given $F_k[i,j]$. Thus, our problem can be solved by finding all the values of $F_n[i,j]$. Although the definition of F_k looks similar to H_k, the following properties make the computation of F_k a better choice over H_k.

Fact 2. *For* $1 \le k \le n$ *and* $1 \le i \le j \le m$,

1. $F_k[i,j] \le F_k[i,j+1]$ if $j+1 \le m$
2. $F_k[i,j] \le F_k[i-1,j]$ if $i \ge 2$
3. $0 \le F_k[i,j] \le 3k$.

Proof. (sketch.) Statement 1 follows directly from Fact 1, and Statement 2 can be proved by induction on j. For Statement 3, it can be proved as follows. In any highest-scoring global alignment of $A[1..k]$ and $B[i..j]$, each character in A or B contribute at least -1 in the alignment score, so $H_k[i,j] \ge -(j-i+1+k)$. This implies $F_k[i,j] \ge 0$. On the other hand, there are at least $|j-i+1-k|$ character-space alignment, and at most k matching-character alignment. Thus $H_k[i,j] \le k - |j-i+1-k|$. This implies that $F_k[i,j] \le k + (k+j-i+1) - |j-i+i-k| = k + 2\min\{k, j-i+1\} \le 3k$.

The above facts imply that the values of F_k are bounded and are monotonic increasing in each row (and decreasing in each column). We next give a related definition, and then show that F_k can be stored in a compact manner.

Definition 9. *For each row* i *of the matrix* F_k*, a* row interval point *is the value* j *such that* $F_k[i,j-1] < F_k[i,j]$*. Note that we assume* $i \le j \le m$*. Similarly, for each column* j *of the matrix, a* column interval points *is the value* i *such that* $F_k[i,j] < F_k[i-1,j]$.

Lemma 2. F_k *can be stored compactly in* $O(km)$ *space. Any value of* $F_k[i,j]$ *can be queried in* $O(\log k)$ *time.*

Proof. For each row i of F_k, we use an array of size $3k+1$ such that the x-th entry stores the smallest j with $F_k[i,j] \ge x$. Note that these values are in essence the row interval points. As there are m rows in F_k, the total space is $O(km)$.

Given such a representation, each value of $F_k[i,j]$ can be found by a binary search in the array corresponding to the i-th row. The query time is $O(\log k)$.

3.2 Computation of F_n

We aim at computing F_n by first computing F_1, and then F_k based on F_{k-1} efficiently for $k = 2, 3, \ldots, n$. For each F_k, instead of computing all the entries, we compute its compact representation as stated in Lemma 2.

We first discuss the computation of the compact form of F_1. For any i, j such that $1 \leq i \leq j \leq m$, under the current score scheme, we have

$$H_1[i,j] = \begin{cases} -(j-i+2) & \text{if } B[i..j] \text{ does not contain } A[1] \\ -(j-i-1) & \text{if } B[i..j] \text{ contains } A[1] \end{cases}$$

Accordingly, we have

$$F_1[i,j] = \begin{cases} 0 & \text{if } B[i..j] \text{ does not contain } A[1] \\ 3 & \text{if } B[i..j] \text{ contains } A[1] \end{cases}$$

Then, to store F_1, we just need to keep an array $L[1..m]$ such that $L[i]$ stores the smallest j such that $B[i..j]$ contains $A[1]$. In fact, $L[i]$ stores the only row interval point of row i of F_1 (Recall that there is one row interval point in each row of F_1). Afterwards, we can determine whether the value of $F_1[i, j']$ is 0 or 3 by comparing j' with $L[i]$, for any j'.

The array L can be computed easily by scanning B once as follows:

1. Initialize all entries of L to 0.
2. Traverse B from $B[1], B[2], \ldots, B[m]$, and set $L[i] = i$ if $B[i] = A[1]$.
3. If $L[m] = 0$, set $L[m] = m + 1$.
4. Examine L backwardly from $L[m-1], L[m-2], \ldots, L[1]$. If $L[k-1] = 0$, set $L[k-1] = L[k]$.

This gives the following lemma.

Lemma 3. *The compact form of F_1 can be computed in $O(m)$ time and $O(m)$ space.*

Next, we show how to compute the compact form of F_k from that of F_{k-1}. Firstly, based on Fact 1 and recursively expand ths F_k terms on the right side, we have an alternative definition of F_k as follows:

$$\begin{aligned} F_k[i,j] &= \max \begin{cases} F_k[i,j-1] \\ F_{k-1}[i,j] \\ F_{k-1}[i,j-1] \ + \ 2 \ + \ \delta(A[k], B[j]) \end{cases} \\ &= \max \begin{cases} F_k[i,j-2] \\ F_{k-1}[i,j-1] \\ F_{k-1}[i,j-2] \ + \ 2 \ + \ \delta(A[k], B[j-1]) \\ F_{k-1}[i,j] \\ F_{k-1}[i,j-1] \ + \ 2 \ + \ \delta(A[k], B[j]) \end{cases} \\ &\ \ \vdots \\ &= \max \begin{cases} \max_{i \leq j' \leq j} F_{k-1}[i,j'] \\ \max_{i+1 \leq j' \leq j} F_{k-1}[i,j'-1] \ + \ 2 \ + \ \delta(A[k], B[j']) \end{cases} \\ &= \max \begin{cases} F_{k-1}[i,j] \\ \max_{i+1 \leq j' \leq j} F_{k-1}[i,j'-1] \ + \ 2 \ + \ \delta(A[k], B[j']) \end{cases} \end{aligned}$$

The above definition is intuitively simpler than that in Fact 1, as each entry of F_k depends only on entries in F_{k-1}. In contrast, the definition in Fact 1 relies on some other entries in F_k itself.

Now, let $G_k[i,j]$ denote $\max_{i+1\le j'\le j} F_{k-1}[i,j'-1]+2+\delta(A[k],B[j'])$, where we assume $G_k[i,j]=0$ if $i=j$. Then, we have

$$F_k[i,j]=\max\{F_{k-1}[i,j],G_k[i,j]\}.$$

The critical part to compute F_k is to compute G_k. For G_k, we observe that

Observation 1. *For any $1\le k\le n$ and $1\le i\le j\le m$,*

1. $G_k[i,j]\le G_k[i,j+1]$ if $j\le m-1$
2. $G_k[i,j]\ge G_k[i+1,j]$ if $i\ge 2$
3. $0\le G_k[i,j]\le 3k$.

Proof. (sketch) Statement 1 follows from definition of G_k, while Statement 2 can be proved by induction on j. For Statement 3, since we have $G_k[i,j]\le F_k[i,j]$ and $G_k[i,j]\ge F_{k-1}[i,j-1]+2+\delta(A[k],B[j])$, it thus follows immediately from Fact 2(3) that $0\le F_k[i,j]\le 3k$.

These observations suggest that G_k can be stored using row interval points. If we have collected all the possible row interval points of G_k, it is easy to see that they can be merged with those of F_{k-1} to get the compact form of F_k, using time linear to the total number of row interval points we consider.

We now claim that in $O(km)$ time, we can find all the possible row interval points of G_k, and the number of which is at most $O(km)$. Suppose that this is true, we have the following results.

Lemma 4. *Given the compact form of F_{k-1}, we can compute the compact form of F_k in $O(km)$ time and $O(km)$ space.*

Theorem 1. *The compact form of F_n can be computed in $O(mn^2)$ time and $O(mn)$ space.*

Proof. Follows directly from Lemmas 3 and 4.

The remaining part focuses on proving the previous claim. Firstly, we show a new property of G_k.

Definition 10. *A* corner point *of the matrix G_k is a tuple (i,j) satisfying*

$$G_k[i,j-1]<G_k[i,j] \text{ and } G_k[i+1,j]<G_k[i,j].$$

Observation 2. *For any corner point (i,j) of G_k, either*

1. *$i=j$, or*
2. *$i+1$ is a column interval point of column j in F_{k-1}.*

Proof. (sketch) All (i,i) are defined to be corner points, as $G_k[i,i-1]$ or $G_k[i+1,i]$ are undefined. For the other corner points (i,j), we have $G_k[i,j] > G_k[i,j-1]$. Then from definitions of $G_k[i,j]$ and $G_k[i,j-1]$, we can derive that $G_k[i,j] = F_{k-1}[i,j-1]+2+\delta(A[k],B[j])$. Now, with $G_k[i+1,j] < G_k[i,j]$, we get $F_{k-1}[i,j-1]+2+\delta(A[k],B[j]) > G_k[i+1,j] \geq F_{k-1}[i+1,j-1]+2+\delta(A[k],B[j])$. This implies $i+1$ is a column interval point of column j in F_{k-1}.

The above shows that the number of possible row interval points for G_k is at most $O(km)$, as there are at most $O(km)$ column interval points in F_{k-1}. The following algorithm completes our claim by showing that the column interval points of F_{k-1} can be found from the row interval points of F_{k-1} in $O(km)$ time and $O(km)$ space.

The algorithm for finding the possible row interval points of G_k is as follows.

1. Examine the compact form, or the row interval points, of F_{k-1}. Compute the *corner points* of F_{k-1}.
2. Sort the corner points (i,j) in the *descending* order of j first, and then in *descending* order of i.
3. Compute the column interval points of F_{k-1} from the corner points of F_{k-1}.
4. Compute the possible column interval points of G_k.
5. Compute the possible row interval points of G_k.

Step 1 can be completed by processing the row interval points of F_{k-1} row by row. Step 2 can be completed by bucket sort. Step 3 can be completed by processing the corner points in the sorted order of Step 2. We use $L[j][x]$ stores the smallest i such that $F_{k-1}[i,j] = x$. Initially, we set all the $O(km)$ entries of L to be 0. When we process (i,j), we let $x = F_{k-1}[i,j]$ and fill $L[j'][x] = i$ for $j' = j, j+1, \ldots$ until it is not equal to 0. Then the array L, and thus the column interval points, will be set correctly. (The correctness is postponed in the full paper.) Step 4 can be completed based on Observation 2. For Step 5, it converts the possible column interval points of G_k to the possible row interval points of G_k, which can be done by a similar approach as Step 1 through Step 3, which converts the row interval points of F_{k-1} to the column interval points. Finally, all the above steps take $O(km)$ time and $O(km)$ space. This completes the proof of the claim.

3.3 Generalizing the Score Scheme

Finally, we consider the general score scheme as follows.

1. $\delta(x,\sqcup) = \delta(\sqcup,x) = -\beta$.
2. $\delta(x,y) = \begin{cases} 1, & \text{if } x = y \text{ and } x,y \neq \sqcup \\ -\alpha, & \text{if } x \neq y \text{ and } x,y \neq \sqcup \end{cases}$

where we consider α and β to be rational numbers.

By setting $F_k[i,j] = H_k[i,j] + (j-i+1+k)\beta + k\alpha$, and using similar tricks as before, we have the following theorem.

Theorem 2. *Under the general score scheme, the All-Substrings Global Alignment Problem can be solved in $O(mn^2)$ time and $O(mn)$ space.*

4 Other Alignment Problems

Given two strings $A[1..n]$ and $B[1..m]$, this section discusses the All-Substrings Alignment Problem for suffix, semi-global, and local alignment. Denote $C_k[i,j]$, $D_k[i,j]$, and $E_k[i,j]$ be the suffix alignment score, the semi-global alignment score and the local alignment score between $A[1..k]$ and $B[i..j]$, respectively. Our problem is to compute $C_k[i,j]$, $D_k[i,j]$, and $E_k[i,j]$ for all $1 \leq k \leq n$ and $1 \leq i \leq j \leq m$. The three lemmas below state the recursive equations for $C_k[i,j]$, $D_k[i,j]$, and $E_k[i,j]$.

Lemma 5.

$$C_k[i,j] = \max \begin{cases} C_k[i,j-1] & + \delta(\sqcup, B[j]) \\ C_{k-1}[i,j] & + \delta(A[k], \sqcup) \\ C_{k-1}[i,j-1] & + \delta(A[k], B[j]) \end{cases}$$

Lemma 6.

$$D_k[i,j] = \max \begin{cases} D_k[i,j-1] & \\ D_{k-1}[i,j] & + \delta(A[k], \sqcup) \\ C_{k-1}[i,j-1] & + \delta(A[k], B[j]) \quad \text{if } A[k] = B[j] \end{cases}$$

Lemma 7.

$$E_k[i,j] = \max \begin{cases} E_k[i,j-1] & \\ E_{k-1}[i,j] & \\ C_{k-1}[i,j-1] & + \delta(A[k], B[j]) \quad \text{if } A[k] = B[j] \end{cases}$$

Using the generalized score scheme as in Section 3.3, C_k, D_k, and E_k satisfy the following lemma.

Lemma 8. *For any i, j, k ($1 \leq k \leq n$,$1 \leq i \leq j \leq m$) such that $j - i \geq k + \lfloor k/\beta \rfloor$,*

1. $C_k[i,j] = C_k[j - \lfloor k/\beta \rfloor - k, j]$
2. $D_k[i,j] = \max_{i \leq i' \leq j-(k+\lfloor k/\beta \rfloor)} D_k[i', i' + (k + \lfloor k/\beta \rfloor)]$
3. $E_k[i,j] = \max_{i \leq i' \leq j-(k+\lfloor k/\beta \rfloor)} E_k[i', i' + (k + \lfloor k/\beta \rfloor)]$

Proof. For Statement 1, consider the suffix alignment between $A[1..k]$ and $B[i..j]$ which achieves the highest score (say, $A[i_1..k]$ with $B[i_2..j]$). Since any suffix of $A[1..k]$ is of length at most k, the suffix alignment contains at most k pairs of matching characters, which contributes at most score k to the alignment. On the other hand, since the alignment score must be at least 0, the suffix alignment can contain at most $\lfloor k/\beta \rfloor$ pairs of character-space match. Thus, $j - i_2 + 1 \leq k + \lfloor k/\beta \rfloor$, and if $j - i \geq k + \lfloor k/\beta \rfloor$, we have $C_k[i,j] = C_k[j - k - \lfloor k/\beta \rfloor, j]$.

Statements 2 and 3 can be proved similarly.

4.1 Computing C_k's

By Lemma 8(1), we observe that many $C_k[i,j]$ values are redundant. Precisely, we only need to compute $\mathcal{C} = \{C_k[i,j] \mid 1 \leq k \leq n \text{ and } j-i \leq k+\lfloor k/\beta \rfloor\}$, while the other entries can be derived by Lemma 8(1). Thus, we have

Lemma 9. *We can compute the values in $\mathcal{C}$ in $O(mn^2)$ time and $O(mn)$ space.*

Proof. For each $k = 1, 2, \ldots, n$, we can apply Lemma 5 to compute those $C_k[i,j]$ with $j-i \leq k+\lfloor k/\beta \rfloor$ in $O(km)$ time and $O(km)$ space. The lemma thus follows.

Lemma 10. *Given the values in $\mathcal{C}$, for any $1 \leq k \leq n$ and $1 \leq i \leq j \leq m$, $C_k[i,j]$ can be retrieved in $O(1)$ time.*

Proof. If $j-i \leq k+\lfloor k/\beta \rfloor$, $C_k[i,j]$ is available in $\mathcal{C}$. Otherwise, if $j-i > k+\lfloor k/\beta \rfloor$, by Lemma 8(1), $C_k[i,j] = C_k[j-(k+\lfloor k/\beta \rfloor), j]$, which is available in $\mathcal{C}$.

4.2 Computing D_k's and E_k's

Similar to C_k, many D_k's and E_k's entries are redundant. We actually require to compute the following two sets of values: $\mathcal{D} = \{D_k[i,j] \mid 1 \leq k \leq n \text{ and } j-i \leq (k+\lfloor k/\beta \rfloor)\}$ and $\mathcal{E} = \{E_k[i,j] \mid 1 \leq k \leq n \text{ and } j-i \leq (k+\lfloor k/\beta \rfloor)\}$. The other D_k and E_k values can be derived using Lemma 8(2) and 8(3). The lemma below indicates that $\mathcal{D}$ and $\mathcal{E}$ can be found in $O(mn^2)$ time.

Lemma 11. *We can compute the values in $\mathcal{D}$ and $\mathcal{E}$ in $O(mn^2)$ time and $O(mn)$ space.*

Proof. By Lemmas 9 and 10, after an $O(mn^2)$ time preprocessing, each $C_k[i,j]$ can be retrieved in $O(1)$ time. Then given C_k, all entries in $\mathcal{D}$ and $\mathcal{E}$ can be computed in $O(mn^2)$ time and $O(mn)$ space, using dynamic programming based on Lemmas 6 and 7.

Given $\mathcal{D}$ and $\mathcal{E}$, the following lemmas imply that $D_k[i,j]$ and $E_k[i,j]$ can be retrieved efficiently after a preprocessing.

Lemma 12. *Given $\mathcal{D}$, we can do an $O(mn)$ time preprocessing so that, for all $1 \leq k \leq n$ and $1 \leq i \leq j \leq m$, $D_k[i,j]$ can be retrieved in $O(1)$ time.*

Proof. For $j-i \leq k+\lfloor k/\beta \rfloor$, the values $D_k[i,j]$'s are available in $\mathcal{D}$ and they can be retrieved in $O(1)$ time.

For $j-i > k+\lfloor k/\beta \rfloor$, by Lemma 8(2), $D_k[i,j] = \max_{i \leq i' \leq j-(k+\lfloor k/\beta \rfloor)} D_k[i', i'+(k+\lfloor k/\beta \rfloor)]$. This is a range maximum query over an array of m numbers. By Lemma 7 of [2], after an $O(m)$ time preprocessing, we obtain an auxiliary data-structure so that $D_k[i,j]$ can be retrieved in $O(1)$ time. We need to do such preprocessing for $k = 1, 2, \ldots, n$. Thus, the total preprocessing time is $O(mn)$.

Lemma 13. *Given $\mathcal{E}$, we can do an $O(mn)$ time preprocessing so that, for all $1 \leq k \leq n$ and $1 \leq i \leq j \leq m$, $E_k[i, j]$ can be retrieved in $O(1)$ time.*

Proof. Similar to Lemma 12.

We conclude this section with the following theorem.

Theorem 3. *The All-Substrings Suffix/Semi-global/Local Alignment Problems can be solved in $O(mn^2)$ time and $O(mn)$ space.*

Proof. The time complexity follows directly from Lemmas 9, 12, and 13. The space complexity follows from the fact that only $O(mn)$ space is needed if we require only the matrices C_n, D_n and E_n.

5 Conclusion

Given two strings $A[1..n]$ and $B[1..m]$, with $n \ll m$, this paper consider the problems of computing the global, local, semi-global, and suffix alignments between A and all m^2 substrings of B. This paper proposes an $O(mn^2)$-time algorithm to solve these problems. Since $n \ll m$, this improves the previous best algorithms which take $O(m^2n)$ time. The improvement stems from a compact representation of all alignment scores.

One future work is to include the affine or the convex gap penalty into the alignment score scheme. Such enhancement is useful and important to biological applications.

References

1. A. V. Aho and T. G. Peterson. A Minimum Distance Error-Correcting Parser for Context-Free Languages. *SIAM Journal on Computing*, 1(4):305–312, 1972.
2. M. A. Bender and M. Farach-Colton. The LCA Problem Revisited. In *Latin American Symposium on Theoretical Informatics*, pages 88–94, 2000.
3. M. W. Du and S. C. Chang. A Model and a Fast Algorithm for Multiple Errors Spelling Correction. *Acta Informatica*, 29(3):281–302, 1992.
4. D. Gusfield. *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology.* Press Syndicate of the University of Cambridge, 1997.
5. J. B. Kruskal. An Overview of Sequences Comparison. In D. Sankoff and J. B. Kruskal, editors, *Time Warps, String Edits and Macromolecules: the Theory and Practice of Sequence Comparison*, pages 1–44. Addison-Wesley, 1983.
6. S. Y. Lu and K. S. Fu. Error-Correcting Tree Automata for Syntactic Pattern Recognition. *IEEE Transactions on Computers*, C-27:1040–1053, 1978.
7. S. B. Needleman and C. D. Wunsch. A General Method Applicable to the Search for Similiarities in the Amino Acid Sequences of Two Proteins. *Journal of Molecular Biology*, 48:443–453, 1970.
8. J. Seitubal and J. Meidanis. *Introduction to Computational Biology.* PWS Publishing Company, 1997.
9. T. F. Smith and M. S. Waterman. Comparison of Biosequences. *Advances in Applied Mathematics*, 2:482–489, 1981.